Sorting Algorithms — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force walkthroughs, and "why we optimize" reasoning for every problem.

1. Selection Sort

Understanding the Problem NEW

You're given a list of numbers in random order, like `[64, 25, 12, 22, 11]`, and you need to rearrange them so they go from smallest to largest. That's it — no tricks, no hidden constraints. The word "sort" just means: put these numbers in increasing order.

Think of it like this: imagine you have 5 playing cards face-up on a table and you want to line them up from lowest to highest, but the rule is you can only work with a physical deck — you look at all the cards, find the smallest one, and place it at the very left. Then you look at the *remaining* cards, find the next smallest, and place it right after. Selection Sort is exactly this process, done by a computer.

ORIGINAL PROBLEM STATEMENT

Sort an array by repeatedly selecting the minimum element and placing it at the beginning.

Brute Force Approach NEW

What would a beginner try first? The most "obvious" way to sort without knowing any algorithm is: repeatedly scan the entire array, and every time you find two adjacent (or any two) elements that are out of order, swap them. Keep doing random passes until nothing is out of order anymore. This is unstructured and inefficient because you have no system — you might swap the same pair back and forth or scan the array many more times than necessary.

A slightly more structured "brute" idea (closer to what Selection Sort formalizes) is: for every position `i` in the array, compare it against *every other* position to check if a smaller element exists anywhere after it, and if so, swap. This still costs roughly `O(n²)` comparisons but without any guarantee of minimal swaps or a clean invariant.

Why this is "brute force": it doesn't use any clever observation about the structure of the problem — it just brute-scans everything repeatedly until the array happens to be sorted.

Why We Optimize NEW

The unstructured brute-force idea has two problems: (1) you don't know *when* to stop, and (2) you may do far more comparisons and swaps than needed because you're not tracking what's already sorted.

The key insight that leads to Selection Sort: **once you know the smallest element in the whole array, you know for certain it belongs at index 0 — permanently.** That single insight lets you split the array into a "sorted zone" (left) and an "unsorted zone" (right), and shrink the unsorted zone by one every round. This turns a vague repeat-until-sorted process into a precise, guaranteed-to-terminate procedure with exactly `n-1` swaps — far fewer writes than naive repeated swapping, which matters when writes (not just comparisons) are expensive, e.g. writing to flash memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [64, 25, 12, 22, 11]

Step 1: Find minimum (11) and swap with first
[11, 25, 12, 22, 64]

Step 2: Find minimum from index 1 (12) and swap
[11, 12, 25, 22, 64]

Step 3: Find minimum from index 2 (22) and swap
[11, 12, 22, 25, 64]

Step 4: Find minimum from index 3 (25) and swap
[11, 12, 22, 25, 64]

Result: [11, 12, 22, 25, 64]
```

The Logic Behind It

Concept: Repeatedly find the smallest element from unsorted part and put it at the beginning.

Step-by-Step Thinking:

1. Find minimum element in unsorted array
2. Swap it with first element of unsorted part
3. Move boundary between sorted and unsorted
4. Repeat until array is sorted

Why This Works:

- Each iteration places one element in correct position
- After i iterations, first i elements are sorted
- Selection sort makes exactly n-1 swaps

Brute Force vs Optimal (original summary)

Brute Force (Naive):

- Compare every element with every other
- $O(n^2)$ time

Optimal (Selection Sort):

- Still $O(n^2)$ but fewer swaps
- $\Theta(n^2)$ comparisons

Algorithm in Theory

```
Algorithm SELECTION_SORT(array):
    For i = 0 to n-1:
        minIndex = i
        For j = i+1 to n-1:
            If array[j] < array[minIndex]:
                 minIndex = j
        Swap array[i] and array[minIndex]
    Return array
```

Pseudocode

```
function selectionSort(arr):
    n = length(arr)
    for i in range(n-1):
        minIdx = i
        for j in range(i+1, n):
            if arr[j] < arr[minIdx]:
                minIdx = j
        swap(arr[i], arr[minIdx])
```

Java Implementation

```
class SelectionSort {
    public void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n-1; i++) {
            int minIdx = i;
            for (int j = i+1; j < n; j++) {
                if (arr[j] < arr[minIdx]) {
```

```
                    minIdx = j;
                }
            }
            swap(arr, i, minIdx);
        }
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Time Complexity

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity

- O(1) - In-place sorting

When to Use Selection Sort

Use When:

- Small arrays
- Memory is limited
- Minimizing writes is important

Avoid When:

- Large arrays
- Performance is critical
- Data is nearly sorted

2. Bubble Sort

Understanding the Problem NEW

Same goal as before — sort a list of numbers — but this time using a different everyday intuition: look at two numbers standing *next to each other*. If the left one is bigger than the right one, they're "out of order," so swap them. Now move one step to the right and repeat, all the way to the end of the array. That's one full "pass."

Why "bubble"? Picture the largest value as a bubble in water — with every pass, it keeps getting compared with its neighbour and, if bigger, keeps moving one step to the right, until it "floats" all the way to the end. After enough passes, every value has bubbled to its correct spot.

ORIGINAL PROBLEM STATEMENT

Sort an array by repeatedly swapping adjacent elements if they are in wrong order.

Brute Force Approach NEW

The most naive version of this idea: keep making full left-to-right passes over the array, swapping any adjacent out-of-order pair you find, and just *always* do exactly `n-1` passes regardless of whether the array became sorted earlier. This works, and is in fact very close to what Bubble Sort already is — but the "naive" version has no early-exit check, so even if the array becomes sorted after pass 2 out of 5, it keeps blindly comparing pairs in passes 3, 4 and 5 that will never need swapping.

Why We Optimize NEW

The wasted work in the naive version comes from not noticing when the job is already done. The optimization is simple but powerful: **track whether any swap happened during a pass.** If an entire pass completes with zero swaps, that means every adjacent pair is already in the right order — the array is fully sorted, and you can stop immediately instead of running the remaining passes.

This single flag (`swapped`) is what turns Bubble Sort's worst case (fully reversed array, always $O(n^2)$) into a best case of just `O(n)` when the input is already sorted or nearly sorted — because you'll do one pass, find zero swaps, and exit.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 1, 4, 2, 8]

Pass 1:
[5, 1, 4, 2, 8] -> [1, 5, 4, 2, 8] (swap 5,1)
[1, 5, 4, 2, 8] -> [1, 4, 5, 2, 8] (swap 5,4)
[1, 4, 5, 2, 8] -> [1, 4, 2, 5, 8] (swap 5,2)
[1, 4, 2, 5, 8] -> [1, 4, 2, 5, 8] (no swap)
After Pass 1: [1, 4, 2, 5, 8]

Pass 2:
[1, 4, 2, 5, 8] -> [1, 2, 4, 5, 8] (swap 4,2)
After Pass 2: [1, 2, 4, 5, 8]

Result: [1, 2, 4, 5, 8]
```

The Logic Behind It

Concept: Repeatedly step through list, compare adjacent elements and swap if they're in wrong order.

Step-by-Step Thinking:

1. Compare adjacent elements
2. Swap if first is greater than second
3. Largest element "bubbles" to end
4. Reduce range and repeat

Why This Works:

- After each pass, largest element in unsorted part reaches end
- With each pass, one element settles in correct position
- If no swaps in a pass, array is sorted

Brute Force vs Optimal (original summary)

Brute Force:

- Compare all pairs
- $O(n^2)$ always

Optimal (Bubble Sort):

- Can stop early if array becomes sorted
- O(n) best case for already sorted array

Algorithm in Theory

```
Algorithm BUBBLE_SORT(array):
    n = length(array)
    For i = 0 to n-2:
        swapped = false
        For j = 0 to n-i-2:
            If array[j] > array[j+1]:
                 Swap array[j] and array[j+1]
                 swapped = true
        If not swapped:
            Break
    Return array
```

Pseudocode

```
function bubbleSort(arr):
    n = length(arr)
    for i in range(n-1):
        swapped = false
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                swap(arr[j], arr[j+1])
                swapped = true
        if not swapped:
            break
```

Java Implementation

```
class BubbleSort {
    public void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n-1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n-i-1; j++) {
                if (arr[j] > arr[j+1]) {
                    swap(arr, j, j+1);
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
```

```
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** O(n²)
- **Worst Case:** O(n²)

Space Complexity

- O(1) - In-place sorting

When to Use Bubble Sort

Use When:

- Small arrays
- Simple implementation needed
- Data is nearly sorted

Avoid When:

- Large arrays
- Performance is important

3. Insertion Sort

Understanding the Problem NEW

This time, imagine sorting playing cards in your hand the way most people actually do it: you hold cards one at a time, and each new card you pick up, you slide it into the correct spot among the cards you're already holding (which are sorted). By the time you've picked up every card, your hand is fully sorted. Insertion Sort is literally that process applied to an array.

ORIGINAL PROBLEM STATEMENT

Sort an array by building the final sorted array one element at a time.

Brute Force Approach NEW

A naive way to "insert" a new element into an already-sorted part of the array is: scan the sorted part from the *beginning*, find the exact index where the new element belongs, then shift everything after that index one place to the right, and drop the element in. This works, but scanning from the beginning every time means you often look at far more elements than necessary, especially if the correct spot is near the end of the sorted part.

Why We Optimize NEW

The optimization here is to scan from the **right end** of the sorted part (the newest neighbour) instead of the left end, and shift elements one-by-one only *while* they're bigger than the element you're inserting. The moment you hit an element that's smaller or equal, you stop — you've found the spot. This means:

- If the array is already nearly sorted, each new element usually needs to move only 0 or 1 positions — giving the excellent `O(n)` best case.
- You never do unnecessary comparisons past the point where the correct position is found.

This is why Insertion Sort is the go-to choice for small or nearly-sorted arrays, and is actually used internally by many real-world sorting libraries (like Java's `Arrays.sort` for small sub-arrays) as a finishing step after another algorithm has done the heavy lifting.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 2, 4, 6, 1, 3]

Step 1: [5] is sorted
[5, 2, 4, 6, 1, 3]

Step 2: Insert 2 into [5]
[2, 5, 4, 6, 1, 3]

Step 3: Insert 4 into [2, 5]
[2, 4, 5, 6, 1, 3]

Step 4: Insert 6 into [2, 4, 5]
[2, 4, 5, 6, 1, 3]

Step 5: Insert 1 into [2, 4, 5, 6]
[1, 2, 4, 5, 6, 3]

Step 6: Insert 3 into [1, 2, 4, 5, 6]
[1, 2, 3, 4, 5, 6]

Result: [1, 2, 3, 4, 5, 6]
```

The Logic Behind It

Concept: Build sorted array by taking one unsorted element and inserting it at correct position.

Step-by-Step Thinking:

1. First element considered sorted
2. Take next element
3. Compare with sorted part from right to left
4. Shift larger elements right
5. Insert element at correct position

Why This Works:

- Maintains invariant: left part is always sorted
- Each element is placed in correct position relative to sorted part
- Shifts are minimal when array is nearly sorted

Brute Force vs Optimal (original summary)

Brute Force:

- Check all positions for each element

- $O(n^2)$

Optimal (Insertion Sort):

- Early exit when position found
- O(n) best case for nearly sorted

Algorithm in Theory

```
Algorithm INSERTION_SORT(array):
    n = length(array)
    For i = 1 to n-1:
        key = array[i]
        j = i - 1
        While j >= 0 and array[j] > key:
            array[j+1] = array[j]
            j = j - 1
        array[j+1] = key
    Return array
```

Pseudocode

```
function insertionSort(arr):
    n = length(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
```

Java Implementation

```
class InsertionSort {
    public void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] = key;
        }
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** O(n^2)
- **Worst Case:** O(n^2)

Space Complexity

- O(1) - In-place sorting

When to Use Insertion Sort

Use When:

- Small arrays
- Nearly sorted data
- Online algorithm needed
- Stable sort required

Avoid When:

- Large arrays
- Reverse sorted data

4. Merge Sort

Understanding the Problem NEW

All three algorithms above share a weakness: in the worst case they take O(n²) time, which becomes painfully slow for large arrays (imagine sorting a million numbers — a million squared is a trillion operations). Merge Sort asks a completely different question: **what if, instead of comparing elements one by one across the whole array, we split the problem into smaller and smaller pieces until each piece is trivially "sorted" (a single element), and then carefully glue the pieces back together in order?**

This "split it up, solve small pieces, combine" strategy is called **divide and conquer**, and it's one of the most important ideas in all of computer science — it shows up again and again in later chapters.

ORIGINAL PROBLEM STATEMENT

Sort array using divide and conquer with guaranteed O(n log n) performance.

Brute Force Approach NEW

The "brute force" comparison point here isn't a naive version of merge sort itself — it's the algorithms we already know (Selection/Bubble/Insertion Sort), which are the natural first attempt anyone makes. They're simple to write, but their O(n²) worst case doesn't scale: doubling the input size roughly quadruples the work.

Why We Optimize NEW

The breakthrough insight: **merging two already-sorted lists into one sorted list is very cheap** — you just walk both lists with two pointers, always taking the smaller of the two current front elements, which costs only O(n) total comparisons for n elements combined. So if we can get our hands on sorted halves cheaply, combining them is fast.

And how do we get sorted halves? By recursively applying the exact same idea to each half, all the way down until a "half" is just one element (which is trivially sorted on its own). Splitting a size-n array in half, over and over, takes only `log n` levels of splitting, and at every level we do O(n) total merging work — giving the famous `O(n log n)` total time, which is *guaranteed* no matter how the input

is arranged (unlike Quick Sort below). The trade-off is that merging needs a temporary array, so Merge Sort costs O(n) extra space — it isn't in-place.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [38, 27, 43, 3, 9, 82, 10]

Divide Phase:
[38, 27, 43, 3, 9, 82, 10]
          v
[38, 27, 43] [3, 9, 82, 10]
     v              v
[38] [27, 43]   [3,9] [82,10]
  v    v          v      v
[38] [27] [43] [3] [9] [82] [10]

Merge Phase:
[38] [27,43] [3,9] [10,82]
     v               v
[27,38,43]     [3,9,10,82]
        v
[3,9,10,27,38,43,82]

Result: [3, 9, 10, 27, 38, 43, 82]
```

The Logic Behind It

Concept: Divide array into halves, sort each half, then merge sorted halves.

Step-by-Step Thinking:

1. If array has 1 element, it's sorted
2. Split array into two halves
3. Recursively sort both halves
4. Merge sorted halves back together

Why This Works:

- Divides problem into smaller subproblems
- Merging two sorted arrays is efficient
- Guarantees O(n log n) regardless of input

Brute Force vs Optimal (original summary)

Brute Force (Insertion Sort):

- $O(n^2)$ worst case

- In-place

Optimal (Merge Sort):

- O(n log n) guaranteed
- Uses extra space

Algorithm in Theory

```
Algorithm MERGE_SORT(array, left, right):
    If left < right:
        mid = (left + right) / 2
        MERGE_SORT(array, left, mid)
        MERGE_SORT(array, mid+1, right)
        MERGE(array, left, mid, right)
    Return array

Algorithm MERGE(array, left, mid, right):
    Create temp arrays leftHalf and rightHalf
    Copy array[left..mid] to leftHalf
    Copy array[mid+1..right] to rightHalf
    i = 0, j = 0, k = left
    While i < len(leftHalf) and j < len(rightHalf):
        If leftHalf[i] <= rightHalf[j]:
            array[k] = leftHalf[i]
            i++
        Else:
            array[k] = rightHalf[j]
            j++
        k++
    Copy remaining elements from leftHalf or rightHalf
```

Pseudocode

```
function mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) / 2
        mergeSort(arr, left, mid)
        mergeSort(arr, mid+1, right)
        merge(arr, left, mid, right)

function merge(arr, left, mid, right):
    leftArr = arr[left..mid]
    rightArr = arr[mid+1..right]
    i = 0, j = 0, k = left

    while i < len(leftArr) and j < len(rightArr):
        if leftArr[i] <= rightArr[j]:
            arr[k] = leftArr[i]
            i++
        else:
```

```
            arr[k] = rightArr[j]
            j++
        k++

    while i < len(leftArr):
        arr[k] = leftArr[i]
        i++; k++

    while j < len(rightArr):
        arr[k] = rightArr[j]
        j++; k++
```

Java Implementation

```
class MergeSort {
    public void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    private void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] leftArr = new int[n1];
        int[] rightArr = new int[n2];

        System.arraycopy(arr, left, leftArr, 0, n1);
        System.arraycopy(arr, mid + 1, rightArr, 0, n2);

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (leftArr[i] <= rightArr[j]) {
                 arr[k] = leftArr[i];
                 i++;
            } else {
                 arr[k] = rightArr[j];
                 j++;
            }
            k++;
        }

        while (i < n1) { arr[k] = leftArr[i]; i++; k++; }
        while (j < n2) { arr[k] = rightArr[j]; j++; k++; }
    }
}
```

Time Complexity

- **Best Case:** O(n log n)
- **Average Case:** O(n log n)
- **Worst Case:** O(n log n)

Space Complexity

- O(n) - Needs extra arrays for merging

When to Use Merge Sort

Use When:

- Guaranteed O(n log n) needed
- Stable sort required
- Linked list sorting
- External sorting

Avoid When:

- Memory is limited
- In-place sorting needed
- Small arrays

5. Recursive Bubble Sort

Understanding the Problem NEW

This is not a new sorting idea — it's the exact same Bubble Sort from problem 2, just written using recursion (a function calling itself) instead of a `for` loop. It's included here mainly as recursion practice: the goal is to express "do one pass, then repeat on a smaller array" as a function calling itself with a shrinking size, instead of a loop counting down.

ORIGINAL PROBLEM STATEMENT

Implement bubble sort using recursion.

Brute Force Approach NEW

There isn't really a separate "brute force vs optimized" split for this one — the underlying algorithm *is* Bubble Sort. The only real choice is implementation style: iterative (loop-based, as in problem 2) vs recursive (function calls itself). The "brute" way a beginner might first write this recursively is to forget the base case or recompute the array length inside every call — leading to either infinite recursion or wasted work.

Why We Optimize NEW

The important thing to get right here is the **base case** and the **shrinking parameter**: each call does one bubble pass over the first `n` elements (pushing the current largest of those `n` to position `n-1`), then recurses with `n-1`. The recursion naturally stops when `n <= 1`, since a single element is already sorted. Unlike the divide-and-conquer recursion in Merge Sort, this recursion doesn't reduce the problem size *exponentially* (halving) — it only reduces it by 1 each time, so the time complexity stays $O(n^2)$, and you additionally pay $O(n)$ extra space for the recursion call stack that the iterative version doesn't need.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 1, 4, 2, 8]
```

```
Recursive call stack:
bubbleSort([5,1,4,2,8], 5)
  -> bubbleSort([1,4,2,5,8], 4) [after one pass]
    -> bubbleSort([1,2,4,5,8], 3) [after two passes]
      -> bubbleSort([1,2,4,5,8], 2) [after three passes]
        -> bubbleSort([1,2,4,5,8], 1) [base case]

Result: [1, 2, 4, 5, 8]
```

The Logic Behind It

Concept: Same as iterative bubble sort but implemented recursively.

Step-by-Step Thinking:

1. If array size is 1, return
2. Perform one pass of bubble sort
3. Recursively call on reduced array size
4. Base case: array of size 1 is sorted

Why This Works:

- After each pass, largest element bubbles to end
- Recursion reduces problem size by 1
- Same logic as iterative version

Algorithm in Theory

```
Algorithm RECURSIVE_BUBBLE_SORT(array, n):
    If n <= 1:
        Return
    For i = 0 to n-2:
        If array[i] > array[i+1]:
            Swap array[i] and array[i+1]
    RECURSIVE_BUBBLE_SORT(array, n-1)
    Return array
```

Pseudocode

```
function recursiveBubbleSort(arr, n):
    if n <= 1:
        return

    for i in range(n-1):
        if arr[i] > arr[i+1]:
            swap(arr[i], arr[i+1])

    recursiveBubbleSort(arr, n-1)
```

Java Implementation

```
class RecursiveBubbleSort {
    public void recursiveBubbleSort(int[] arr, int n) {
        if (n <= 1) return;

        for (int i = 0; i < n-1; i++) {
            if (arr[i] > arr[i+1]) {
                swap(arr, i, i+1);
            }
        }
        recursiveBubbleSort(arr, n-1);
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** O(n^2)
- **Worst Case:** O(n^2)

Space Complexity

- O(n) - Recursion stack space

When to Use Recursive Bubble Sort

Use When:

- Learning recursion
- Simple implementation needed

Avoid When:

- Performance is critical
- Deep recursion concerns

6. Recursive Insertion Sort

Understanding the Problem NEW

Same relationship as the previous problem: this is Insertion Sort (problem 3), rewritten recursively. The clever bit is *which* part gets the recursive call: instead of recursing on "the rest of the array to insert," it recurses on "sort everything except the last element first," and only *after* that recursive call returns does it insert the last element into its correct spot.

ORIGINAL PROBLEM STATEMENT

Implement insertion sort using recursion.

Brute Force Approach NEW

As with recursive bubble sort, there's no separate "brute force" algorithm here — it's about recursion style. A naive attempt might try to recurse "forward" (sort element 0, then recurse on the rest), which actually breaks the natural insertion-sort invariant. The version given here recurses "backward" (assume the first n-1 are already sorted via recursion, then insert the n-th).

Why We Optimize NEW

The reason the recursion is structured as "recurse first, insert after" is that it mirrors the iterative version's invariant exactly: **"the first k elements are always sorted."** By recursing down to `n=1` first (trivially sorted), then inserting element by element as the recursive calls return (unwind), each insertion has a properly sorted left neighbourhood to slot into — exactly like the iterative loop, just expressed with the call stack doing the "looping" instead of a `for` statement. The time complexity remains $O(n^2)$ since insertion itself is unchanged; the only new cost is O(n) recursion stack space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [5, 2, 4, 6, 1, 3]

Recursive call stack:
```

```
insertionSort([5,2,4,6,1,3], 0) [base case]
  -> insertionSort([5,2,4,6,1,3], 1)
    -> insertionSort([5,2,4,6,1,3], 2)
      -> insertionSort([5,2,4,6,1,3], 3)
        -> insertionSort([5,2,4,6,1,3], 4)
          -> insertionSort([5,2,4,6,1,3], 5) [actual sorting starts here]

Result: [1, 2, 3, 4, 5, 6]
```

The Logic Behind It

Concept: Same as iterative insertion sort but implemented recursively.

Step-by-Step Thinking:

1. Base case: array of size 1 is sorted
2. Recursively sort first n-1 elements
3. Insert last element in correct position
4. Shifts elements right as needed

Why This Works:

- Recursion handles sorting first n-1 elements
- Insertion step places last element correctly
- Same logic as iterative version

Algorithm in Theory

```
Algorithm RECURSIVE_INSERTION_SORT(array, n):
    If n <= 1:
        Return
    RECURSIVE_INSERTION_SORT(array, n-1)
    key = array[n-1]
    j = n-2
    While j >= 0 and array[j] > key:
        array[j+1] = array[j]
        j = j-1
    array[j+1] = key
    Return array
```

Pseudocode

```
function recursiveInsertionSort(arr, n):
    if n <= 1:
        return

    recursiveInsertionSort(arr, n-1)

    key = arr[n-1]
```

```
    j = n-2
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j--
    arr[j+1] = key
```

Java Implementation

```
class RecursiveInsertionSort {
    public void recursiveInsertionSort(int[] arr, int n) {
        if (n <= 1) return;

        recursiveInsertionSort(arr, n-1);

        int key = arr[n-1];
        int j = n-2;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

Time Complexity

- **Best Case:** O(n) - Already sorted
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity

- O(n) - Recursion stack space

When to Use Recursive Insertion Sort

Use When:

- Learning recursion
- Simple implementation needed
- Small arrays

Avoid When:

- Performance is critical
- Deep recursion concerns

7. Quick Sort

Understanding the Problem NEW

Quick Sort asks: can we get Merge Sort's O(n log n) speed *without* paying for the extra O(n) memory that merging needs? The idea: instead of blindly splitting the array in half and merging afterward, pick one element (called the **pivot**) and rearrange (*partition*) the array so that everything smaller than the pivot ends up on its left, and everything bigger ends up on its right. Once that's done, the pivot is already sitting in its final, correct sorted position — no merging step required. Then just repeat the same trick separately on the left group and the right group.

ORIGINAL PROBLEM STATEMENT

Sort an array efficiently using divide and conquer.

Brute Force Approach NEW

The natural comparison point is Merge Sort: it guarantees O(n log n) but always needs a temporary array to merge into, which costs extra memory and extra copying. A "brute" alternative to picking a good pivot is to always pick, say, the first or last element as pivot without any thought. This works correctly, but if the input happens to already be sorted (or reverse-sorted), always picking an end element as pivot is the *worst possible* choice — it causes maximally unbalanced partitions.

Why We Optimize NEW

The whole performance story of Quick Sort comes down to **how balanced the partition is**. If the pivot happens to land near the middle of the sorted order every time, the array roughly halves at each level (just like Merge Sort), giving `O(n log n)` — but this time *in-place*, with no extra array, because partitioning rearranges elements within the same array.

But if you always pick a bad pivot (e.g. the smallest or largest element, which happens naturally on already-sorted input if you always choose the last element), each partition only peels off one element instead of splitting in half — degrading to `O(n²)`, the same as the simple sorts we started with. This is exactly why real implementations use a **random pivot** or **median-of-three** selection (shown at the end of this problem): it makes the worst case astronomically unlikely in

> practice, so Quick Sort behaves like O(n log n) almost all the time while staying in-place — which is why it tends to outperform Merge Sort in practice despite having a worse theoretical worst case.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Visual Diagram

```
Array: [50, 30, 80, 20, 70, 10, 60, 40]

Step 1: Choose pivot (last element: 40)
[50, 30, 80, 20, 70, 10, 60, 40]
              Pivot=40

Partition: Arrange elements around pivot
Elements < 40: [30, 20, 10]
Elements > 40: [50, 80, 70, 60]
After partition: [30, 20, 10, 40, 50, 80, 70, 60]

Step 2: Recursively sort left and right
Left: [30, 20, 10] Right: [50, 80, 70, 60]

Step 3: Sort left part
[30, 20, 10] -> pivot=10
Partition: [10, 20, 30]
Sort sub-parts: [10], [20], [30] -> [10, 20, 30]

Step 4: Sort right part
[50, 80, 70, 60] -> pivot=60
Partition: [50, 60, 70, 80]
Sort sub-parts: [50], [70, 80] -> [70, 80]
Final: [50, 60, 70, 80]

Result: [10, 20, 30, 40, 50, 60, 70, 80]
```

The Logic Behind It

Concept: Choose a pivot, place it in its correct position, and recursively sort the elements on both sides.

Step-by-Step Thinking:

1. Choose Pivot: Select an element as pivot
2. Partition: Rearrange array so all elements < pivot are on left, all elements > pivot are on right, and pivot is in its correct position
3. Recursively Sort: Apply same process to left and right subarrays

Why This Works:

- After partitioning, pivot is in correct position
- Left subarray contains smaller elements
- Right subarray contains larger elements
- Each step reduces problem size

Visual Animation

```
Step 1: Choose pivot
[50, 30, 80, 20, 70, 10, 60, 40]
                              ^
                            pivot

Step 2: Partition
i = -1
[50, 30, 80, 20, 70, 10, 60, 40]
  ^
  i

Compare 50 > 40 -> No swap
[50, 30, 80, 20, 70, 10, 60, 40]
      ^
      i

Compare 30 < 40 -> Swap with i+1
[30, 50, 80, 20, 70, 10, 60, 40]
  ^
  i

Compare 80 > 40 -> No swap
Compare 20 < 40 -> Swap with i+1
[30, 20, 80, 50, 70, 10, 60, 40]
      ^
      i

Compare 70 > 40 -> No swap
Compare 10 < 40 -> Swap with i+1
[30, 20, 10, 50, 70, 80, 60, 40]
         ^
         i

Compare 60 > 40 -> No swap
Place pivot in position i+1
[30, 20, 10, 40, 70, 80, 60, 50]
             ^
           pivot at position 3
```

Brute Force vs Optimal (original summary)

Brute Force (Merge Sort):

- O(n log n) but uses extra space
- Always divides exactly in half

Optimal (Quick Sort):

- O(n log n) average
- In-place sort
- Faster in practice due to better cache performance

Algorithm in Theory

```
Algorithm QUICK_SORT(array, low, high):
    If low < high:
        pivotIndex = PARTITION(array, low, high)
        QUICK_SORT(array, low, pivotIndex-1)
        QUICK_SORT(array, pivotIndex+1, high)
    Return array

Algorithm PARTITION(array, low, high):
    pivot = array[high]
    i = low - 1
    For j = low to high-1:
        If array[j] <= pivot:
            i = i + 1
            Swap array[i] and array[j]
    Swap array[i+1] and array[high]
    Return i+1
```

Pseudocode

```
function quickSort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)

function partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j from low to high-1:
        if arr[j] <= pivot:
            i += 1
            swap(arr[i], arr[j])
    swap(arr[i+1], arr[high])
    return i+1
```

Java Implementation

```
class QuickSort {
    public void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    private int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, high);
        return i + 1;
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Time Complexity Explanation

Why O(n log n) Average?

Best Case (Pivot always splits array in half):

```
T(n) = 2T(n/2) + O(n)      -> T(n) = O(n log n)
```

Worst Case (Pivot always min or max):

```
T(n) = T(n-1) + O(n)       -> T(n) = O(n²)
```

Average Case:

- Assuming random pivot, splits are balanced on average
- T(n) = O(n log n)

Time Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n²)

Why Worst Case O(n²)?

- If pivot is always smallest/largest
- Each partition only removes one element
- Causes O(n²) comparisons

Space Complexity Explanation

Why O(log n) Average?

Quick sort:

- In-place sort (no extra array)
- Recursion stack space

Recursion Stack:

- Best/Average: O(log n) - Balanced partitions
- Worst Case: O(n) - Unbalanced partitions

Memory Usage:

```
Input array: n elements
Recursion stack: O(log n) average
Total: O(n) + O(log n)
```

When to Use Quick Sort

Use When:

- Average case performance is good
- In-place sort is needed
- Cache performance matters
- Pivot selection is random

Avoid When:

- Worst-case performance is critical
- Array is already sorted (can use randomization)
- Stable sort is required

Pivot Selection Strategies

```
// 1. First element pivot
int pivot = arr[low];

// 2. Last element pivot (used above)
int pivot = arr[high];

// 3. Middle element pivot
int mid = (low + high) / 2;
int pivot = arr[mid];

// 4. Random pivot
int randomIndex = low + random.nextInt(high - low + 1);
int pivot = arr[randomIndex];

// 5. Median of three
int mid = (low + high) / 2;
int pivot = median(arr[low], arr[mid], arr[high]);
```

Comparison Summary

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Recursive Bubble	O(n)	O(n²)	O(n²)	O(n)	Yes
Recursive Insertion	O(n)	O(n²)	O(n²)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No

— End of enhanced Chapter 1: Sorting Algorithms — remaining chapters continue in the full edition —